Coding & Solution

Date	23 October 2026
Team ID	0827CY231029
Project Name	Waypoint - AI Travel Itinerary Planner
Maximum Marks	5 Marks

Coding & Solution

The implementation combines a React/Vite frontend with a modular Python/FastAPI backend. The source includes configuration, database schemas and local/deployment guides.

Solution Summary

Field	**Details**
Repository Link / URL	https://github.com/2praveen6/travel-itinerary-skill-wallet
Programming Language(s)	Python; JavaScript/JSX; CSS; SQL.
Framework(s) Used	FastAPI, Pydantic, React 18, Vite and React Router.
Key Features Implemented	Authentication; preferences; curated templates; custom generation; daily timelines; saved-trip management; share links; JSON/Markdown/ICS export; recommendations.
Pending / Incomplete Features	Multi-city trips, simultaneous editing, offline access and native mobile app. Formal load testing and a recorded public demo are not supplied.
Setup / Run Instructions	From 5. Project Development Phase/backend: install requirements and run uvicorn app.main:app --reload --port 5000. From frontend: npm install; npm run dev. Open localhost:5173.

Coding & Solution (continued)

Code Quality Checklist

S.No	Criteria	Status	Evidence / Notes
1	Modular functions / classes	Yes	Routers, schemas, service modules and repository adapters are separate.
2	Meaningful variable and function names	Yes	Examples include generate_itinerary, clone_template and get_current_user.
3	Comments / documentation where necessary	Partial	README and architecture/setup/API guides exist; comment coverage is uneven.
4	Error handling for critical operations	Partial	HTTP/validation handlers and AI fallback exist; some raw exception messages reach responses.
5	Application runs without critical errors	Not verified	This document is based on source inspection; a fresh runtime verification was not performed.
6	Version control repository	Yes	A local Git repository and GitHub repository link are present; current working files include uncommitted changes.

Additional Notes / Comments

Refinement accepts a requested day number in the prompt, but the current endpoint replaces the returned plan; unchanged-day preservation should be tested. Provider selection and local-rule fallback should be described separately from an automatic retry chain.

Project sources: backend routers/, services/, main.py and security.py; frontend src/; project documentation and local Git status.